Catalogue Management System — How Everything Works

Oct 5, 2026 · @Rupam

1. The big picture

In one sentence: the Catalogue Management System is a B2B (business-to-business) product catalogue and marketplace. Suppliers bring products in (file upload, API or streaming), buyers browse and order, sellers approve orders, and payments, analytics and an AI assistant sit on top.

Think of it as five layers, each with one job:

Layer	What it is	Folder	Local address
Frontend	The screens people click	`web/catalogue-ui` (Vue 3 + Vite)	http://localhost:5173
Gateway	One front door that forwards requests	`src/Catalogue.Gateway` (YARP)	http://localhost:5000
API	Business rules exposed as HTTP endpoints	`src/Catalogue.Web` + Application + Domain + Infrastructure	http://localhost:5080
Background work	Jobs that run with nobody waiting	`src/Catalogue.Workers`, `src/Catalogue.Functions`, `logicapps/`	worker process, Functions on :7072
Data and services	Where things are stored or sent	SQL Server LocalDB, Azurite/disk blobs, Redis, Service Bus, Event Hubs, Foundry Local	your machine / Docker

How one request travels

```
Browser (Vue SPA :5173)
  | axios call with header  Authorization: Bearer <JWT>
  v
Vite proxy /api -> API (default), or Gateway (YARP :5000) when
API_TARGET=http://localhost:5000
  v
API (Catalogue.Web :5080)
  middleware : correlation id -> error handler -> authentication -> rate
limit -> timeout
  endpoint   : e.g. GET /api/products   (minimal API in ProductEndpoints.cs)
  service    : Application layer (business rules)
  data       : EF Core -> SQL Server     cache: HybridCache (memory + Redis)
  files      : Blob storage (Azurite ./.azurite  or disk ./.storage)
  messages   : Service Bus queue / Event Hubs  -> Worker / Functions pick
them up
  v
JSON response -> back through the gateway -> Vue updates the screen
```

Key idea to say out loud: every layer can be swapped for its Azure twin (SQL Server to Azure SQL, Azurite to Azure Storage, Foundry Local to Azure OpenAI) mostly by changing configuration, because the code talks to interfaces, not to concrete machines. Section 13 shows exactly how.

2. Glossary: every abbreviation in plain words

Read this once; the rest of the document uses these words freely.

General web and .NET

Term	Stands for	Simple meaning	Where in this project
API	Application Programming Interface	A set of URLs another program can call to get or change data	`src/Catalogue.Web/Endpoints`
REST	Representational State Transfer	A style of API: nouns in the URL (`/api/products/5`), verbs from HTTP (GET, POST, PUT, DELETE)	all endpoints
HTTP	HyperText Transfer Protocol	The language browsers and servers speak	everywhere

Term	Stands for	Simple meaning	Where in this project
JSON	JavaScript Object Notation	Text format for data: `{"name":"Bolt","price":2.5}`	request and response bodies
DTO	Data Transfer Object	A small class that only carries data in or out of an endpoint, so the database entity is never exposed directly	`Catalogue.Application` records
DI	Dependency Injection	The framework creates objects and hands them to whoever needs them, so classes ask for an interface instead of building things themselves	`Program.cs` `builder.Services.Add...`
Minimal API	(no abbreviation)	Endpoints written as `app.MapGet("/x", handler)` instead of controller classes	`Endpoints/*.cs` (see docs/api)
Middleware	(no abbreviation)	A step every request passes through, like a security checkpoint	`CorrelationIdMiddleware`, exception handler
ProblemDetails	RFC 7807	Standard JSON shape for errors (`title`, `status`, `detail`)	`GlobalExceptionHandler`
CORS	Cross-Origin Resource Sharing	Browser rule deciding which websites may call the API	`AllowedOrigins` in appsettings
SPA	Single Page Application	One web page that swaps views with JavaScript instead of reloading	the Vue app
SSE	Server-Sent Events	Server keeps a response open and pushes small messages (used to stream the AI answer live)	agent run stream endpoint
YARP	Yet Another Reverse Proxy	Microsoft library that forwards requests to other servers	`Catalogue.Gateway`

Security

Term	Stands for	Simple meaning	Where
JWT	JSON Web Token	A signed ticket proving who you are; valid 30 minutes here	`Authorization: Bearer ...`
Refresh token		Long-lived secret in the `cat_rt` cookie used to get a new JWT; rotated on every use, and reuse of an old one revokes the family	Auth endpoints
PBKDF2	Password-Based Key Derivation Function 2	Slow hashing so stolen password hashes are hard to crack	password hasher
HMAC	Hash-based Message Authentication Code	A signature made with a secret key; proves a URL or webhook was not tampered with	signed media URLs, Stripe webhook
SSRF	Server-Side Request Forgery	An attack that tricks the server into calling internal addresses; we block private IPs when downloading images	media import guard

Term	Stands for	Simple meaning	Where
Rate limiting		Max number of calls per time window (login, register, agent: 30/hour/user)	`Program.cs` policies
RBAC / Policy	Role-Based Access Control	Only certain roles may call certain endpoints (Admin, Supplier, Buyer, Seller)	`RequireAuthorization(Policies.X)`

Data

Term	Stands for	Simple meaning	Where
SQL	Structured Query Language	Language for relational databases	SQL Server
LocalDB	SQL Server Express LocalDB	A tiny SQL Server that starts on demand on your PC	default connection string
ORM	Object-Relational Mapper	Turns C# classes into tables and LINQ into SQL	EF Core
EF Core	Entity Framework Core	Microsoft's ORM	`CatalogueDbContext`
LINQ	Language Integrated Query	`db.Products.Where(p => p.Price > 10)` style queries in C#	services
Index		A sorted lookup so the database does not scan every row	e.g. `Products.CreatedAtUtc`

Term	Stands for	Simple meaning	Where
MERGE		One SQL statement that inserts new rows and updates existing ones (an "upsert")	bulk import
SqlBulkCopy		Very fast way to push thousands of rows into a table	bulk import
Cache / HybridCache		Keeps recent answers in memory (L1) and Redis (L2) so the database is not asked again; entries tagged so they can be cleared together	product lists, analytics
Redis	Remote Dictionary Server	Fast in-memory key/value store shared by all API copies	L2 cache

Product and business words

Term	Stands for	Simple meaning
B2B	Business to Business	Companies selling to companies
SKU	Stock Keeping Unit	A seller's own product code
GTIN	Global Trade Item Number	The barcode number (EAN/UPC)
MPN	Manufacturer Part Number	The maker's code for the part
MOQ	Minimum Order Quantity	You must buy at least this many
Pack size		Sold in multiples of this (quantities get rounded up)
PO	Purchase Order	One order to one seller; a basket with 3 sellers becomes 3 POs
KYB	Know Your Business	Checking a company is real before it can sell

Term	Stands for	Simple meaning
EDI / PRICAT	Electronic Data Interchange / PRICe CATalogue message	Old but common standard file format suppliers use to send price lists
CSV	Comma-Separated Values	Spreadsheet-like text file for imports

Cloud and messaging

Term	Simple meaning
Blob	Binary Large OBject: any file (image, PDF, CSV) stored in a container, like a folder in the cloud
Azurite	Microsoft's free emulator of Azure Storage that runs on your PC; our data lives in `./.azurite` inside the project
Queue / Service Bus	A mailbox for jobs: the API drops a message ("process import 42"), a worker picks it up later. Survives crashes and retries
Event Hubs	A high-speed conveyor belt for streams of events (millions per second); readers remember their position with **checkpoints**
Azure Functions	Small pieces of code that run when something happens (a timer, an HTTP call, a message) without managing a server
Durable Functions	An extension of Functions for long, multi-step workflows that **remember where they were** even if the machine restarts. Explained fully in section 7
Orchestrator	The Durable "manager" function: decides the order of steps. Must be deterministic (same input gives same decisions) because it is replayed
Activity	The Durable "worker" function: does the real work (call an API, write to DB). Can be retried
Client function	Starts an orchestration (from a timer or an HTTP call)

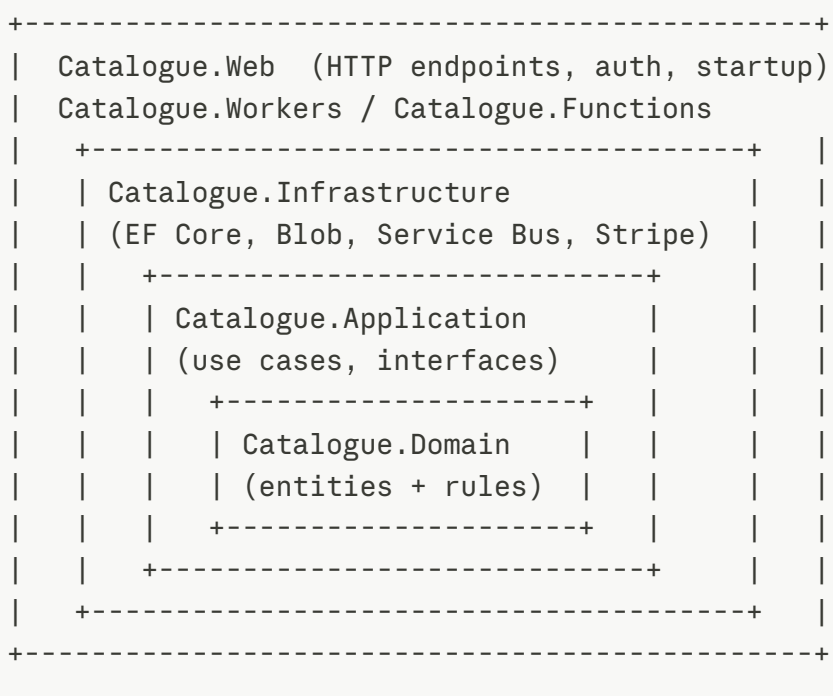
Term	Simple meaning
Fan-out / fan-in	Start many tasks in parallel (fan-out), then wait for all of them (fan-in)
Logic Apps	Drag-and-drop (JSON) workflows: "every 5 minutes, list failed imports, then send an email"
ACR	Azure Container Registry: a private store for Docker images
ACA	Azure Container Apps: runs containers for you, scales them, gives them HTTPS
Docker image / container	An image is a packaged app with everything it needs; a container is a running copy of it
Managed Identity	An identity Azure gives your app so it can log in to other Azure services without any password in config
Key Vault	Azure's safe for secrets (connection strings, JWT key, Stripe key)
App Insights	Application Insights: Azure's logs, traces and metrics dashboard
OTel	OpenTelemetry: open standard for traces, metrics and logs; we send them to the Aspire dashboard locally
Polly	.NET library for retries, timeouts and circuit breakers when calling other services
Serilog	Structured logging library (logs as data, not just text)
CI / CD	Continuous Integration / Delivery: GitHub Actions builds and tests every push; CD would deploy it

AI

Term	Simple meaning
LLM	Large Language Model: the AI that reads and writes text
Foundry Local	Microsoft tool that downloads and runs LLMs on your own PC with an OpenAI-compatible API (we use `ministral-3-3b-instruct-2512`)
Microsoft Agent Framework	Library for building AI agents: a model plus instructions plus tools plus memory, and multi-step workflows
Agent	An LLM that can call **tools** (C# functions) to look up real data before answering
Tool / function calling	The model replies "call `find_products` with `{query:'bolt'}`"; our code runs it and gives the result back
Token	A piece of a word; models have limits on how many they read and write
Human-in-the-loop	The agent can only *propose* changes; a person must approve before anything is written

3. How the solution is organised

The rule: inner layers know nothing about outer layers. This is called **Clean Architecture** (also "onion architecture"). The business rules in the middle do not care whether data lives in LocalDB or Azure SQL, so the outside can be swapped.



Project	Job in one line	Important files
`Catalogue.Domain`	The nouns and their rules: Product, Supplier, PurchaseOrder, Payment, AgentActionProposal. No database code	`Entities.cs`, `MarketplaceDomain.cs`, `IdentityDomain.cs`, `MediaDomain.cs`, `AgentDomain.cs`
`Catalogue.Application`	The verbs (use cases): import a file, price a basket, take a payment, compute analytics. Declares **interfaces** like `ICatalogueDbContext`, `IFileStorage`, `IPaymentGateway`	`ImportProcessor.cs`, `Marketplace.cs`, `Pricing.cs`, `Payments.cs`, `Analytics.cs`, `Abstractions.cs`
`Catalogue.Infrastructure`	Real implementations of those interfaces: EF Core, Blob/disk storage, Service Bus, Event Hubs, CSV/EDI parsers, Stripe	`Persistence/CatalogueDbContext.cs`, `Storage/*`, `Messaging/ImportQueues.cs`, `Payments/PaymentGateways.cs`, `DependencyInjection.cs`
`Catalogue.Contracts`	Message shapes shared between API and workers (e.g. `ImportRequested`)	`Messages.cs`

Project	Job in one line	Important files
`Catalogue.ServiceDefaults`	Shared plumbing every service uses: OpenTelemetry, health checks, correlation id, error handler	`ServiceDefaults.cs`, `Correlation.cs`, `GlobalExceptionHandler.cs`
`Catalogue.Web`	The API host: `Program.cs` wires everything, `Endpoints/` exposes 107 endpoints, `Security/` handles JWT and identity	`Program.cs`, `Endpoints/*.cs`, `Security/Identity.cs`
`Catalogue.Agents`	The AI layer: Foundry Local client, tools, agent runs, operations-review workflow	`CatalogueAssistant.cs`, `Tools/`, `Review/`, `FoundryLocal/`
`Catalogue.Gateway`	YARP reverse proxy, the single front door	`Program.cs`, `appsettings.json` routes
`Catalogue.Workers`	Background process that reads the import queue	`ImportWorker.cs`
`Catalogue.Functions`	Azure Functions + Durable Functions for supplier sync	`SupplierSyncFunctions.cs`, `host.json`
`logicapps/`	Logic Apps Standard workflows for import monitoring and notification	`import-monitor/workflow.json`, `import-notify/workflow.json`
`web/catalogue-ui`	Vue 3 single page app	`src/api.ts`, `src/router.ts`, `src/views/*`
`tests/`	xUnit unit tests (172) and worker tests (6)	`Catalogue.UnitTests`, `Catalogue.WorkerTests`
`docs/api`	One page per endpoint file, minimal API vs controller	`README.md`, `01-auth.md` to `14-agent-analytics.md`

Why this shape helps you explain the project

- **Testable:** the Application layer can be tested with fake implementations of its interfaces (that is what the unit tests do).
- **Swappable:** moving to Azure means writing (or configuring) a new class in Infrastructure; Domain and Application do not change.
- **Readable:** each endpoint file is short because the real logic lives in a service.

4. Backend coding (the API)

Definition: the backend is the program on the server that receives HTTP requests, checks who is calling, applies business rules, talks to the database and returns JSON. Ours is an **ASP.NET Core (.NET 10) minimal API** in `src/Catalogue.Web`.

4.1 Startup: `Program.cs` in four steps

Step 1: register services (DI). We tell the DI container which class to create for each need.

```
var builder = WebApplication.CreateBuilder(args);
builder.AddServiceDefaults("catalogue-api");            // Serilog,
OpenTelemetry, error handler, health
builder.Services.AddCatalogueInfrastructure(builder.Configuration,
builder.Environment.ContentRootPath);
builder.Services.AddCataloguePayments(builder.Configuration);
builder.Services.AddCatalogueAgents(builder.Configuration);
builder.Services.AddScoped<ImportService>();
builder.Services.AddScoped<MarketplacePricing>();
builder.Services.AddScoped<AnalyticsService>();
```

`AddScoped` = one instance per HTTP request. `AddSingleton` = one for the whole app. `AddTransient` = a new one each time it is asked for.

Step 2: cross-cutting protection.

Feature	Code	Why
HybridCache	`AddHybridCache(...)` (+ Redis if `ConnectionStrings:Redis` is set, wrapped in `ResilientDistributedCache`)	Fast repeated reads; a Redis outage never breaks a request
Request timeouts	default 30 s, `LongRunning` 15 min	A slow call returns 504 instead of hanging forever
Rate limiting	Login 10/min per IP, Register 5/hour per IP, Agent 30/hour per user	Stops brute force and protects the GPU
JWT authentication	`AddJwtBearer` with issuer + signing key checks	Every call proves who it is

Feature	Code	Why
Authorization policies	`CatalogueAdmin`, `SupplierAdmin`, `ImportOps`, `ReadCatalogue`, `Buyer`, `Seller`, `Marketplace`	Role-based access
CORS	`AllowedOrigins` (default `http://localhost:5173`)	Only our frontend may call from a browser
Health checks	database, eventhub, local-model	`/health` tells ops what is broken

Step 3: middleware pipeline (order matters).

```
app.UseServiceDefaults();   // correlation id + global exception handler
app.UseCors();
app.UseRateLimiter();
app.UseAuthentication();    // who are you? (reads the JWT)
app.UseAuthorization();     // are you allowed?
app.UseRequestTimeouts();
```

Step 4: seed the database and map endpoints. `db.SeedAsync()` creates the schema and demo data, a "recovery sweep" marks imports stuck in `Processing` as failed (so they can be reprocessed after a crash), then every endpoint file is mapped under `/api`:

```
var api = app.MapGroup("/api");
Catalogue.Web.Endpoints.AuthEndpoints.Map(api);
Catalogue.Web.Endpoints.ProductEndpoints.Map(api);
// ... 15 files, 107 endpoints in total
```

4.2 What an endpoint looks like

A real one from `Endpoints/CategoryEndpoints.cs`:

```
api.MapGet("/categories", async (ICatalogueDbContext db, HybridCache cache,
CancellationToken ct) =>
    Results.Ok(await cache.GetOrCreateAsync("categories:all",
        async token => await db.Categories.AsNoTracking()
            .OrderBy(c => c.SortOrder).ThenBy(c => c.Name)
            .Select(c => new CategoryDto(c.Id, c.Code, c.Name,
c.ParentCategoryId, c.IsActive, c.SortOrder))
            .ToListAsync(token),
        new HybridCacheEntryOptions { Expiration = TimeSpan.FromMinutes(10)
},
        [CacheTags.Categories], ct)))
    .RequireAuthorization("ReadCatalogue").WithTags("Categories");
```

Read it as a sentence: *"For GET /api/categories, ask DI for the database and the cache; return the cached list if we have it, otherwise query, shape each row into a `CategoryDto`, cache it for 10 minutes under the tag `Categories`; only people allowed to read the catalogue may call it."* When a category is created, the code calls `cache.RemoveByTagAsync(CacheTags.Categories)` so the next read is fresh.

Minimal API vs controller: a controller would be a class `CategoriesController : ControllerBase` with `[HttpGet]` methods and constructor injection. Same result, more ceremony. The full side-by-side for all 107 endpoints is in `docs/api/`.

4.3 Where business rules live

Endpoints stay thin; they call services in `Catalogue.Application` and methods on Domain entities. Example: an order's state can only change through methods such as `PurchaseOrder.TransitionTo(next, note)` or `MarkPaid(now)`, which refuse illegal moves (for example, a Shipped order cannot go back to Submitted). This is called a **rich domain model**: the rule lives next to the data, so no endpoint can forget it.

4.4 Authentication flow (login to API call)

1. `POST /api/auth/login` with email + password. Rate-limited.
2. Server checks the PBKDF2 hash (`PasswordHasher<AppUser>`, 100k iterations).
3. Server returns a **JWT** (30 min) in the body and sets a **refresh token** in an HttpOnly cookie `cat_rt` (JavaScript cannot read it, so XSS cannot steal it).
4. The SPA keeps the JWT **in memory only** and sends it as `Authorization: Bearer ...`.

5. When the JWT expires, the SPA calls `POST /api/auth/refresh`; the cookie is rotated. If an *old* refresh token is ever reused, the whole token family is revoked (someone stole it).

4.5 Errors, logs and tracing

- **GlobalExceptionHandler** turns any unhandled exception into a ProblemDetails JSON with a `traceId`, never a stack trace.
- **CorrelationIdMiddleware** gives each request an `X-Correlation-Id`, copied into logs, audit events and queue messages, so one id follows a request across API, worker and Functions.
- **Serilog** writes structured logs to console and `logs/`. **OpenTelemetry** sends traces and metrics to the Aspire dashboard (locally) or Application Insights (Azure).
- **Polly** (via `AddStandardResilienceHandler`) retries outgoing HTTP calls with backoff and opens a circuit breaker if a dependency keeps failing; EF Core uses `EnableRetryOnFailure` for SQL blips and deadlocks (error 1205).

4.6 The gateway

`Catalogue.Gateway` is a **YARP reverse proxy** on :5000. The browser calls one address; YARP forwards `/api/*` to the API. In Azure this lets you scale the API to many copies, add more backends later, or swap it for Azure API Management / Front Door without touching the frontend.

5. Database coding

Definition: the database is where everything permanent is kept: products, prices, users, orders, payments, imports, audit history. We use **SQL Server** (LocalDB on your PC) and talk to it through **EF Core**, an ORM that turns C# classes into tables.

5.1 From class to table

1. **Entity** (Domain): a plain C# class, e.g. `Product` with `Sku`, `Name`, `Brand`, `Gtin`.
2. **DbSet** (Infrastructure): `CatalogueDbContext` lists every table, 29 of them:

```
public sealed class CatalogueDbContext(DbContextOptions<CatalogueDbContext>
options)
    : DbContext(options), ICatalogueDbContext
{
    public DbSet<Product> Products => Set<Product>();
    public DbSet<PurchaseOrder> PurchaseOrders => Set<PurchaseOrder>();
    public DbSet<Payment> Payments => Set<Payment>();
    // ... Suppliers, Categories, ProductPrices, Inventories, ImportFiles,
Users, RefreshTokens, AgentConversations ...
}
```

3. **Mapping** (`OnModelCreating`): lengths, keys, relationships and indexes, written in C#:

```
modelBuilder.Entity<Product>(b =>
{
    b.Property(p => p.Sku).HasMaxLength(64).IsRequired();
    b.HasIndex(p => new { p.SupplierId, p.Sku }).IsUnique(); // a supplier
cannot repeat a SKU
    b.HasIndex(p => p.CreatedAtUtc);                          // analytics:
products added per day
});
```

4. **Interface** (Application): `ICatalogueDbContext` exposes the same DbSets, so services depend on the interface, not on SQL Server.
5. **Registration** (`DependencyInjection.cs`):

```
services.AddDbContext<CatalogueDbContext>(o => o.UseSqlServer(connStr, sql =>
{
    sql.EnableRetryOnFailure(maxRetryCount: 5, maxRetryDelay:
TimeSpan.FromSeconds(10), errorNumbersToAdd: [1205]);
    sql.CommandTimeout(60);
}));
```

Default connection string: `Server=(localdb)\mssqllocaldb;Database=CatalogueDb;Trusted_Connection=True;TrustServerCertificate=True`.

5.2 Main tables, grouped

Area	Tables
Catalogue	Products, Categories, ProductCategories, AttributeDefinitions, CategoryAttributes, ProductAttributeValues, ProductAssets
Supply and price	Suppliers, SupplierOffers, ProductPrices, Inventories, PriceLists, PriceListEntries, SupplierDiscounts, CustomerAccounts
Imports	ImportFiles, ImportItemErrors, ImportStreamChunks, MediaFetchRequests, ProductChangeRequests
Identity	Organizations, Users, RefreshTokens
Marketplace	PurchaseOrders, PurchaseOrderLines, Payments
AI	AgentConversations, AgentMessages, AgentActionProposals, OperationsReviews
History	AuditEvents (who did what, with correlation id)

5.3 How the schema is created (and why not migrations, yet)

On startup `db.SeedAsync()` runs:

1. `EnsureCreatedAsync()` creates the database and all tables if the database does not exist.
2. **`SchemaEvolver.EvolveAsync()`** compares the EF model with the live database and **adds** any missing table, column or index. It never drops anything, so it is safe on every start.
3. `ApplySchemaPatchesAsync()` runs idempotent SQL (`IF NOT EXISTS ... CREATE INDEX`) and tunes locking for parallel imports (row locks, no lock escalation).
4. Seed data: categories, demo suppliers, HVAC demo products, admin/demo users.

Honest note for interviews: this is a learning-friendly shortcut. Production uses **EF Core migrations** (`dotnet ef migrations add`, then `dotnet ef database update` or a migration bundle in the release pipeline). Section 13 shows the switch.

5.4 Reading data well

- `AsNoTracking()` for read-only queries: faster, EF does not watch the objects for changes.
- Project to DTOs with `.Select(...)` so only needed columns are fetched.
- Paging with `Skip/Take` and an `X-Total-Count` header.
- **Sargable filters** (index-friendly). Example fix we made: the "has image" filter was rewritten as an `EXISTS` on an indexed column, taking catalogue pages from ~20 s to ~23 ms.
- HybridCache in front of expensive queries (category tree, analytics totals for 10 min).

5.5 Writing data safely

- **Unit of work:** change entities, then one `SaveChangesAsync()` commits all of it in one transaction.
- **Constraints in the database, not just code:** check constraint `[Amount] >= 0` on prices; unique `(SupplierId, Sku)`.
- **Filtered unique indexes** enforce business rules even under races:
  - `Payments(PurchaseOrderId) WHERE Status IN ('Succeeded','Refunded')` means an order can never be paid twice.
  - `AgentActionProposals(Kind, TargetId) WHERE Status = 'Pending'` means only one open AI proposal per target.
- **Audit trail:** important writes add an `AuditEvent` in the same transaction.

5.6 Bulk writing millions of rows

EF Core is too slow for a 2-million-row supplier file, so `SqlBulkCatalogueWriter` does it set-based:

1. Create temp tables `#rows`, `#attrs`, `#media`.
2. `SqlBulkCopy` streams the chunk into them in one round trip.
3. `MERGE dbo.Products AS t USING #rows ...` inserts new products and updates existing ones in one statement; same for Inventories and SupplierOffers.
4. Record the chunk in `ImportStreamChunks (ImportId, ChunkIndex)` **inside the same transaction**. If an Event Hub event is delivered twice, the chunk is already in the ledger and is skipped. That is **idempotency**: doing it twice has the same effect as once.

6. File handling (uploads, Blob storage, images)

Definition: file handling is everything to do with bytes that are not rows in a table: supplier CSV/EDI files, product images, PDFs, ZIP media packs. Files go into **Blob storage**; the database only keeps a pointer (path, size, hash).

6.1 Two kinds of files, two interfaces

Interface (Application)	Holds	Blob container	Disk fallback folder
`IFileStorage`	Raw supplier import files (CSV, EDI)	`catalogue-raw`	`.storage/files`
`IMediaStorage`	Product images, thumbnails, documents	`catalogue-media`	`.storage/media`

```
public interface IFileStorage
{
    Task<string> SaveAsync(Stream content, string fileName, CancellationToken
ct);
    Task<Stream> OpenReadAsync(string path, CancellationToken ct);
}
```

Each interface has two implementations in `Infrastructure/Storage`: `BlobFileStorage` / `BlobMediaStorage` (Azure SDK `BlobContainerClient`) and `LocalFileStorage` / `LocalMediaStorage` (plain disk).

6.2 Choosing Blob or disk at startup

`StorageSelection.Resolve` (in `DependencyInjection.cs`) decides once at startup:

1. Is `ConnectionStrings:BlobStorage` set and `FileStorage:UseBlob` true? (Locally it is `UseDevelopmentStorage=true`, which means Azurite on 127.0.0.1:10000.)
2. Is the Blob endpoint reachable within 3 seconds? Yes: use Blob. No: log a warning and use disk.
3. Either way, data stays **inside the project folder**: Azurite stores its data in `./.azurite`, the disk fallback uses `./.storage` (`Storage:Root`, default `../../.storage` from the API folder). Nothing goes to `%TEMP%`.

Blob uploads use exponential retry (6 tries, up to 30 s apart, 5-minute network timeout for large files) and containers are created **private** (`PublicAccessType.None`).

6.3 Supplier import file flow

```
POST /api/imports (multipart: file + supplierCode + format)
  -> IFileStorage.SaveAsync  -> raw/2026/10/05/<guid>-prices.csv
  -> ImportFile row (Status = Received, StoragePath, Format)
  -> IImportQueue.PublishAsync(ImportRequested{ImportId, CorrelationId})
  <- 202 Accepted  (the user does not wait)

Worker (LocalImportWorker in the API, or Catalogue.Workers)
  -> IImportQueue.ReceiveAsync
  -> IFileStorage.OpenReadAsync  (stream, never the whole file in memory)
  -> ISupplierFileParser (CsvSupplierParser or EdiSupplierParser) yields rows
one by one
  -> RowValidator: bad rows -> ImportItemErrors (row number + message)
  -> good rows -> upsert products / prices / stock
  -> ImportFile.Status = Completed | CompletedWithErrors | Failed
```

Very large EDI files (up to 2 GB) use the **streaming path** instead: the file is split into chunks and published to Event Hubs (section 7).

6.4 Product image flow (`MediaService.AddAsync`)

validate, de-duplicate, store, thumbnail, record:

1. **Magic-byte sniffing:** `MediaSniffer.Detect` looks at the first bytes of the file (for example JPEG starts `FF D8 FF` , PNG starts `89 50 4E 47`). The file name extension is not trusted, so a virus renamed to `.jpg` is rejected.
2. **Size limit:** images up to 10 MB.
3. **SHA-256 hash** of the bytes: the same image uploaded twice for a product is stored once.
4. **Store** the original, then a **320 px JPEG thumbnail** made with SkiaSharp (transparent PNGs are flattened onto white).
5. **Record** a `ProductAsset` row: path, thumbnail path, content type, size, width, height, hash, who uploaded it.

Bulk options: upload a ZIP whose file names match SKU/MPN (`BulkMediaImporter`), or let supplier feeds reference image URLs, which `MediaFetchService` downloads in the background. Before downloading, `UrlGuard` resolves the host and **refuses private and**

loopback IPs (SSRF protection).

6.5 Serving files safely: signed URLs

Images must load in an `<img>` tag, which cannot send a JWT header. So the API gives out **signed, expiring URLs**:

```
// MediaUrlSigner
return $"/api/media/{assetId}/{variant}?exp={exp}&sig={Sign(assetId, variant, exp)}";
// Sign = HMAC-SHA256(key, "{id}:{variant}:{exp}")
```

`GET /api/media/{id}/{o|t}` recomputes the signature and compares it in constant time (`FixedTimeEquals`). Wrong or expired signature: 403. It also sends an **ETag** (from the hash) so browsers can cache images. In Azure, this same idea is a **SAS token** (Shared Access Signature) on the blob URL.

7. Background work: queues, streaming, Durable Functions, Logic Apps

Why background work exists: some jobs take minutes (a 2 GB supplier file, syncing 50 suppliers). A web request should answer in milliseconds, so the API **accepts** the job, stores it, returns `202 Accepted`, and something else does the work. We use four tools, each for a different shape of job:

Tool	Shape of job	Our use	Code
Queue (in-memory or Service Bus)	One message = one job	"Process import 42"	`Messaging/ImportQueues.cs`, `ImportWorker.cs`, `LocalImportWorker.cs`
Event Hubs	Huge stream of small events, processed in parallel	2 GB EDI file split into chunks	`Streaming/*`, `EventHubIngestionService`

Tool	Shape of job	Our use	Code
Durable Functions	Long multi-step workflow with retries and fan-out	Sync every REST supplier every 30 min	`Catalogue.Functions/SupplierSyncFunctions.cs`
Logic Apps	Low-code integration and notifications	Watch for failed imports, notify	`logicapps/*/workflow.json`

7.1 Import queue

Definition: a queue is a mailbox. Producers drop messages in; consumers take them out one at a time. If the consumer crashes, the message is still there.

```
public interface IImportQueue
{
    ValueTask PublishAsync(ImportRequested message, CancellationToken ct);
    ValueTask<ImportRequested?> ReceiveAsync(CancellationToken ct);
}
```

- **Local:** `InMemoryImportQueue` uses a bounded `Channel<T>` (1,000 slots) inside the API process; `LocalImportWorker` (a `BackgroundService` in the API) reads it. Simple, but lost on restart, which is why startup runs a **recovery sweep** that marks stuck imports as failed and reprocessable.
- **Azure:** set `ServiceBus:Enabled=true` and `ServiceBusImportQueue` is used instead. Messages get `MessageId = import-{id}` (Service Bus **duplicate detection** drops repeats), `CorrelationId` for tracing, and **PeekLock** receive: the message is locked, then `CompleteMessageAsync` on success or `AbandonMessageAsync` on failure so it is retried (and eventually moved to the **dead-letter queue**). The separate `Catalogue.Workers` process (a .NET Worker Service) consumes it, so you can scale workers independently of the API.

7.2 Event Hubs streaming

Definition: Event Hubs is a log of events split into **partitions**. Many readers work in parallel, one per partition, and each remembers how far it has read with a **checkpoint**.

Our flow for a big EDI (PRICAT) file:

1. Upload: `EdiStreamService` streams the file to Blob while hashing it (SHA-256), with a 2 GB cap.
2. `EdiStreamPublisher` reads EDI segments and publishes `EdiChunkEvent`s (a few thousand products each) to the hub.
3. `EventHubIngestionService` runs an `EventProcessorClient` per worker; partitions are balanced across all running workers automatically.
4. For each event, `EdiChunkProcessor` uses the bulk writer (SqlBulkCopy + MERGE, section 5.6) and records the chunk in the `ImportStreamChunks` ledger.
5. `args.UpdateCheckpointAsync()` saves progress to Blob storage. After a crash, reading resumes from the last checkpoint; the ledger makes replayed chunks harmless.
6. `StreamTelemetry` drives the live progress screen (`EventHubView.vue`).

Locally `EventHub:Enabled=false` uses `InMemoryEventHub` (same interface, same code path, in one process).

7.3 Durable Functions (the key definition)

Azure Functions = small functions Azure runs for you when a trigger fires (timer, HTTP, queue). You do not manage servers; you pay per execution.

Durable Functions = an extension that lets you write a **long-running, stateful workflow as ordinary C# code**. Azure saves the workflow's progress (an **event-sourced history**) in storage after every step. If the machine restarts halfway, the workflow **replays** its history and continues where it stopped instead of starting again.

Three roles:

Role	Simple meaning	Rules
Client	Starts a workflow	Any trigger: timer, HTTP, queue

Role	Simple meaning	Rules
Orchestrator	The manager: decides what happens next	Must be **deterministic**: no `DateTime.Now`, no direct I/O, no random. Use `ctx.CurrentUtcDateTime`, call activities instead
Activity	The worker: does real I/O (DB, HTTP)	Can be retried with a policy

Our implementation (`SupplierSyncFunctions.cs`):

```
SyncTimerClient  [TimerTrigger "0 */30 * * * *"]  (skips if a sync is already
running)
SyncHttpClient   [HttpTrigger POST /api/sync/start] -> returns instanceId +
status URL
        |
        v
SupplierSyncOrchestration
   1. GetRestSuppliersActivity           -> list of active REST suppliers
   2. FAN-OUT: one SingleSupplierSyncOrchestration per supplier (in parallel)
        a. FetchSupplierProductsActivity  (retry 3x: 5s, 10s backoff)
        b. split into chunks of 25
        c. FAN-OUT: UpsertChunkActivity per chunk (each retried on its own)
        d. FAN-IN: sum upserted counts
   3. FAN-IN: Task.WhenAll -> SyncSummary
   4. AuditSyncActivity                   -> AuditEvent
"SupplierSyncCompleted"
```

The heart of it:

```
private static readonly TaskOptions RetryPolicy = new(new RetryPolicy(
    maxNumberOfAttempts: 3, firstRetryInterval: TimeSpan.FromSeconds(5),
backoffCoefficient: 2.0));

[Function(nameof(SupplierSyncOrchestration))]
public static async Task<SyncSummary>
SupplierSyncOrchestration([OrchestrationTrigger] TaskOrchestrationContext
ctx)
{
    var suppliers = await ctx.CallActivityAsync<List<SupplierRef>>
(nameof(GetRestSuppliersActivity), (object?)null);
    var results = await Task.WhenAll(suppliers.Select(s =>
        ctx.CallSubOrchestratorAsync<SupplierSyncResult>
(nameof(SingleSupplierSyncOrchestration), s, RetryPolicy)));
    var summary = new SyncSummary(suppliers.Count, results.Sum(r =>
r.Fetched), results.Sum(r => r.Upserted), results.Sum(r => r.Errors));
    await ctx.CallActivityAsync(nameof(AuditSyncActivity), summary);
    return summary;
}
```

Where state is kept: `host.json` sets `durableTask.storageProvider = azureStorage` using `AzureWebJobsStorage`. Locally that is `UseDevelopmentStorage=true` (Azurite); in Azure it is a real Storage account (tables hold history, queues carry work items). `maxConcurrentActivityFunctions = 8` limits parallel activities per instance.

Why Durable, not a simple loop? A loop inside one function would time out, lose progress on restart, and need hand-written retries. Durable gives checkpointing, per-step retry, parallel fan-out and a status URL for free.

Run locally: `cd src/Catalogue.Functions` then `func start --port 7072`, then `POST http://localhost:7072/api/sync/start`.

7.4 Logic Apps (Standard)

Definition: Logic Apps are workflows made of a **trigger** and **actions**, stored as JSON and edited visually in the Azure portal. Good for gluing services together (HTTP, email, Teams, Service Bus) without writing C#.

Workflow	Trigger	Steps
`import-notify`	HTTP request `{ "importId": N }`	Log in to the Catalogue API, get the import status, return status, row counts and correlation id
`import-monitor`	Recurrence, every 5 minutes	List imports; for each `Failed` / `Rejected` one, fetch row-level errors (in Azure: send a Teams/email alert here)

Locally they run with Azure Functions Core Tools (`cd logicapps; func host start`) and Azurite. CI validates the workflow JSON on every push.

8. Business features, end to end

Each feature below follows the same path: **Vue view, then endpoint file, then service, then domain rule, then table.**

Feature	Screen (Vue)	Endpoints	Service / rule
Sign up, login, companies	`LoginView`, `RegisterView`, `OrganizationsView`	`AuthEndpoints`, `OrganizationEndpoints`	`TokenService`, `Organization.Approve/Reject` (KYB)
Catalogue browse and edit	`CatalogueView`, `ProductEditView`, `CategoriesView`	`ProductEndpoints`, `CategoryEndpoints`	EF queries + HybridCache
Supplier imports	`ImportsView`, `ImportDetailView`, `SuppliersView`	`ImportEndpoints`, `SupplierEndpoints`	`ImportService`, `ImportProcessor`, parsers
Big-file streaming	`EventHubView`	`EventHubEndpoints`	`EdiStreamService`, Event Hubs
Approvals	`ApprovalsView`	`ApprovalEndpoints`	`ApprovalService`
B2B pricing	`PricingView`	`PricingEndpoints`	`PriceResolver`, `PricingService`
Images and documents	`MediaView`, `MediaManager`	`MediaEndpoints`	`MediaService`, `BulkMediaImporter`

Feature	Screen (Vue)	Endpoints	Service / rule
Marketplace basket and orders	`BasketView`, `OrdersView`, `SellerView`	`MarketplaceEndpoints`	`MarketplacePricing`, `SourcingService`, `OrderStatusService`
Payments	`OrdersView` payment panel, `PaymentsView`	`PaymentEndpoints`	`PaymentService`, `IPaymentGateway`
Dashboard and audit	`DashboardView`, `AuditView`	`DashboardEndpoints`	aggregate queries
Analytics and AI	`AnalyticsView`, `AssistantView`, `OperationsView`	`AnalyticsEndpoints`, `AgentEndpoints`	section 9

8.1 Organisations and roles (KYB)

A company registers, its status is `PendingVerification`, and a platform admin approves it (`Active`) or rejects it. Approval can link it to a Supplier (so it can sell) and/or a CustomerAccount (so it gets B2B prices). Roles: **Admin, SupplierManager, Buyer, Seller, Support, Viewer**; each endpoint names the policy it needs.

8.2 Approvals (four-eyes check)

Supplier feeds should not change the live catalogue blindly. A **new product** or a **price change bigger than the supplier's threshold** (`PriceChangeApprovalThresholdPercent`) becomes a `ProductChangeRequest`. An admin approves (product goes live, or old price is closed and the new one starts) or rejects (the catalogue stays as it was). Bulk approve works on a filter, in batches.

8.3 B2B pricing

`PriceResolver` explains every price it gives. Order of rules:

1. The customer's own price list, then its parents (**customer, then tier, then trade**), up to 10 levels, stopping on cycles.
2. On each list, the explicit entry with the **highest quantity break** the buyer qualifies for wins.
3. No entry anywhere? The first list with a **percentage discount** applies to the list price.
4. No lists at all? The list price stands.

8.4 Marketplace: basket to purchase orders

Definition: a marketplace lets many sellers offer the same product. `MarketplacePricing.CompareAsync` prices **every** seller's offer for this buyer:

- **base** = the offer's selling price (or the product's list price)
- **platform** = the buyer's price-list rules (section 8.3)
- **supplier** = the best `SupplierDiscount` that seller gives this buyer
- quantity is **rounded up to MOQ and pack size**; a company never buys from itself.

The basket lives in the browser (`basket.ts` , localStorage). On checkout `SourcingService.PlaceAsync` **re-prices everything on the server** (never trusts prices from the browser) and creates **one Purchase Order per seller**.

Order life cycle, enforced by `PurchaseOrder.TransitionTo` :

```
Submitted --> Accepted --> Shipped
    |            |
    +--> Rejected +--> Cancelled
    +--> Cancelled
```

Anything else throws ("A shipped order cannot be cancelled").

8.5 Payments

Definition: a payment gateway is a service that moves money (cards, wallets). We support two, behind one interface `IPaymentGateway` :

Gateway	What it does	When it is on
Simulated	Shows a fake card dialog and only updates the database. No money moves	`Payments:Simulated:Enabled=true` (Development)
Stripe Checkout (test mode)	Redirects to Stripe's hosted page; test cards like 4242 4242 4242 4242	`Payments:Stripe:SecretKey` = an `sk_test_...` key. Live keys (`sk_live_`) are refused unless `AllowLiveKeys`

Flow:

1. `POST /api/marketplace/orders/{id}/pay` creates a `Payment` (Pending) and asks the gateway for a checkout URL. A second click **reuses** the pending attempt instead of creating another.
2. The buyer pays on Stripe (or in the simulated dialog).
3. Stripe calls our **webhook** `POST /api/payments/webhooks/stripe`. We verify the `Stripe-Signature` **HMAC** and reject anything older than 5 minutes (replay protection). The return page also calls `GET /api/payments/{id}?sync=true`, which asks Stripe for the latest state, so payment works even if the webhook is late.
4. `Payment.Succeed()` is **idempotent** (calling it twice changes nothing) and the order becomes `Paid`. The filtered unique index guarantees one successful payment per order, even if webhook and sync race.
5. If the seller rejects or the buyer cancels a paid order, `PaymentService.RefundAsync` refunds through the same gateway and marks it `Refunded`.

Orders can also be placed with no gateway configured; they then settle "on account" (invoice), like real B2B trade.

9. AI agent and analytics

9.1 What the agent is

Definition: an **AI agent** is a language model that can **use tools**. Instead of guessing, it says "call `list_orders` with `{status:'Submitted'}`", our C# runs that function against the real database, gives the result back, and the model writes the answer. We build it with the **Microsoft Agent Framework** (`Microsoft.Agents.AI`, version 1.23) and run the model **on your own GPU** with **Foundry Local**, so no data leaves the machine and there is no per-token bill.

Piece	File	Job
Model runtime	`FoundryLocal/FoundryLocalRuntime.cs`	Finds Foundry Local (its port changes each start, so we ask `foundry server status -o json`), loads the model if needed. Endpoint must be loopback
Chat client	`FoundryLocal/FoundryLocalChatClients.cs`	OpenAI-compatible client + fixes for small models (e.g. Ministral crashes on `content:null`, so empty assistant turns get a space)

Piece	File	Job
Tool-call parser	`FoundryLocal/TextToolCallParser.cs`	Small models write tool calls as text in different dialects (Qwen `<tool_call>` XML, Hermes JSON, Mistral `name{json}`); this turns them into real function calls
Tools	`Tools/CatalogueAgentTools.cs`	9 tools: `operations_overview`, `find_products`, `product_details`, `list_orders`, `order_details`, `list_imports`, `pending_approvals`, `business_metrics`, `propose_action`
Assistant	`CatalogueAssistant.cs`	`ChatClientAgent` with instructions, tools, and chat memory trimmed by `MessageCountingChatReducer`
Runs	`Runs/AgentRuns.cs`	Each question is a background **run**; the browser follows it live over **SSE**
Review workflow	`Review/OperationsReviewWorkflow.cs`	A multi-step **workflow**: Collect, then Triage, then Briefing, then Publish

Default model: `ministral-3-3b-instruct-2512` (3.6 GB, fast: about 5 s for a two-tool question). Optional: `qwen3.5-9b` (better reasoning, much slower).

9.2 Safety rules built in

- **Same permissions as the user:** tools run as an `AgentPrincipal` built from the caller's JWT. A Buyer's agent can only see that buyer's orders.
- **Read-only tools + proposals:** the agent can never write directly. `propose_action` creates an `AgentActionProposal` (Pending, expires in 3 days). A human clicks Approve; `AgentActionExecutor` **re-checks the approver's rights** and only then performs the change.
- **Budgets:** `ToolBudget` caps tool calls per run; runs time out (20 min); rate limit 30 agent calls per hour per user.
- **Guarded functions:** each tool is wrapped (`GuardedFunction`) so an exception becomes a clean error message to the model instead of crashing the run.

9.3 A question, step by step

```
AssistantView.vue  -> POST /api/agent/conversations/{id}/messages  -> 202 {
runId }
                   -> GET  /api/agent/runs/{runId}/events   (SSE stream)
AgentRunWorker     -> FoundryLocalRuntime.EnsureReadyAsync (load model)
                   -> ChatClientAgent.RunStreamingAsync
                        model: "call list_orders{...}"  -> event: tool_call
                        C#: CatalogueAgentTools.ListOrders -> event:
tool_result
                        model: text tokens             -> event: delta (typed
out live)
                   -> save AgentMessages, publish event: done
```

9.4 Operations review workflow

A scheduled (or on-demand) **workflow** built with `WorkflowBuilder` and four **executors**:

1. **Collect:** gathers numbers in C# (failed imports, pending approvals, unpaid orders), no AI.
2. **Triage:** ranks issues by rules.
3. **Briefing:** the model writes a short briefing by calling a `submit_briefing` tool (structured output, so it cannot ramble).
4. **Publish:** saves an `OperationsReview` row shown on `OperationsView`.

Why a workflow? Each step is testable, and only one step needs the model.

9.5 Analytics dashboard

Definition: analytics = turning raw rows into charts that answer "how are we doing?".

- `AnalyticsService` (Application) builds one `AnalyticsWindow` per filter: **today, 7 days, 15 days, 1 month, 3 months, 6 months, 1 year**. It picks the bucket size automatically (hour, day, week, month) and also computes the **previous period** so the UI can show "+12% vs last period".
- Endpoint `GET /api/analytics?range=7d` (long-running timeout). Data is **scoped**: platform staff see everything, a company sees only its own orders and products. Product daily counts are cached 10 minutes.
- Charts are our own SVG Vue components (no chart library): `PulseAreaChart` (the "breathing" animated line/area), `GradientBars` (bar graph), `DonutChart` (pie), `RadialGauge`, `Sparkline`.

- First load on millions of rows is about 25 s; cached loads are 0.3 to 1.2 s.

10. Frontend creation (Vue 3 SPA)

Definition: the frontend is the part that runs **in the browser**. Ours is a **Single Page Application**: the browser downloads one `index.html` plus JavaScript once, then swaps screens without reloading, fetching data from the API as JSON.

Tool	What it does here
Vue 3	UI framework. Each screen is a `.vue` file with `<template>` (HTML), `<script setup lang="ts">` (logic) and `<style>`
TypeScript	JavaScript with types, so mistakes show up at build time (`vue-tsc`)
Vite	Dev server with instant reload on :5173, and the production bundler (`vite build` creates `dist/`)
vue-router	Maps URLs (`/orders`) to views and guards them
axios	HTTP client used for every API call
Pinia	Installed for shared state; most state is in small reactive modules (`session.ts` , `basket.ts` , `ui.ts`)

10.1 How the app starts (`main.ts`)

```
applyTheme(theme.value)
// Restore the session from the httpOnly refresh cookie before the router's
first auth check.
restoreSession().finally(() =>
createApp(App).use(createPinia()).use(router).mount('#app'))
```

So on page reload the user stays logged in without the token ever being in localStorage.

10.2 Folder map

File / folder	Job
`src/api.ts`	The **only** place that talks to the backend: axios instance, interceptors, and typed functions + TypeScript interfaces for every endpoint (also the SSE client `followRun` for the AI assistant)
`src/session.ts`	Who is logged in, JWT in memory, and `session.can('buyer')`, which mirrors the server's policies so the UI only shows what the user may do
`src/router.ts`	Routes with `meta: { auth, policy, title, section }`; views are **lazy-loaded** (`() => import(...)`) so the first download is small
`src/basket.ts`	Marketplace basket kept in localStorage (no secrets, just product ids and quantities)
`src/ui.ts`	Theme (light/dark) and toast messages
`src/views/*`	24 screens (Catalogue, Imports, Orders, Analytics, Assistant…)
`src/components/*`	Reusable parts: `Pager`, `StatusBadge`, `MediaManager`, `Overlay`, `EmptyState`, and `charts/*`

10.3 The API client: what every request gets for free

```
export const api = axios.create({ baseURL: '/api', timeout: 60_000 })

api.interceptors.request.use(config => {
  if (session.token) config.headers.Authorization = `Bearer ${session.token}`
  config.headers['X-Correlation-ID'] ??= newCorrelationId()   // one id per
user action
  return config
})
```

The response interceptor adds:

1. **401 handling:** refresh the token once through the cookie (`/api/auth/refresh` , single-flight so ten parallel 401s cause one refresh), then replay the request; if that fails, go to `/login?expired=1` .
2. **Retries:** only for safe methods (GET/HEAD/OPTIONS) on network errors or 408/429/502/503/504, up to 2 times with **exponential backoff + jitter**, honouring `Retry-After` .
3. **Proactive refresh:** every 30 s, if the token expires within 2 minutes, refresh it so active users never see a 401.

10.4 Route guard

```
router.beforeEach(to => {
  if (to.meta.auth && !session.isAuthenticated) return { path: '/login',
query: { next: to.fullPath } }
  if (to.meta.policy && !session.can(to.meta.policy)) return '/catalogue'
})
```

This is for user experience only. **Real security is on the server**: even if someone edits the JavaScript, the API still checks the JWT and policy.

10.5 How a screen is built (pattern used by every view)

1. `const rows = ref<Order[]>([])` , `loading` , `error` refs.
2. `onMounted(load)` , where `load()` calls a typed function from `api.ts` .
3. Template renders a table/cards, `StatusBadge` for states, `Pager` for paging (reads the `X-Total-Count` header), `EmptyState` when nothing is found.
4. Actions (approve, pay, upload) call the API, show a toast, and reload.
5. Errors show the **correlation id**, so support can find the exact server log line.

10.6 Charts without a chart library

The analytics charts are hand-written SVG components: data is mapped to coordinates with `computed` , and CSS animations create the "breathing" effect (`PulseAreaChart`), gradient fills (`GradientBars`), pie (`DonutChart`), gauge (`RadialGauge`) and mini trend lines (`Sparkline`). Colours come from `charts/palette.ts` and follow light/dark theme.

10.7 Dev vs production

	Development	Production (container)
Served by	Vite dev server :5173	**nginx** serving `dist/` (`web/catalogue-ui/Dockerfile`: node build stage, then `nginx:alpine`)
`/api` goes to	Vite proxy to `http://localhost:5080` (or the gateway with `API_TARGET=http://localhost:5000`)	nginx `location /api/` proxies to `http://api:8080`, 2 GB uploads, 600 s timeout
Deep links	handled by Vite	`try_files $uri /index.html` so `/orders/5` works after refresh

Because the browser always calls the same origin (`/api`), there is **no CORS** problem in either mode.

11. Testing and CI

Definition: a **unit test** is a small piece of code that calls one piece of our code with known input and checks the output, automatically, in milliseconds. **CI** (Continuous Integration) runs all tests on every push so a mistake is caught before it reaches users.

11.1 What we test

178 tests pass: 172 in `Catalogue.UnitTests` and 6 in `Catalogue.WorkerTests`. Built with **xUnit**, using **EF Core InMemory** as a fake database (fast, no SQL Server needed).

Test file	Proves that…
`CsvParserTests`, `EdiParserTests`	Supplier files are parsed into the right rows, bad rows are reported with row numbers
`ImportServiceTests`, `ImportProcessorTests`, `HvacImportTests`	Upload, queue and process work end to end; errors are recorded

Test file	Proves that…
`HvacDomainTests`	Product rules: GTIN check digits, attributes, change requests
`MarketplaceTests`	Price chain, supplier discounts, MOQ / pack rounding, one PO per seller, legal order transitions
`PaymentTests`	Simulated and Stripe flows, webhook signature and 5-minute window, no double success, refunds, live keys refused
`MediaTests`	Magic-byte detection, signed URL verify/expiry, SSRF guard blocks private IPs
`StreamingTests`	EDI chunking, chunk ledger makes replays harmless
`ResilienceAndCorrelationTests`	Correlation id flows through, cache failures do not break requests
`AgentTests`	Tool-call parsing for each model dialect, tool budgets, permissions, proposals, workflow steps (with a fake chat client, no GPU needed)

A typical test reads like a sentence (Arrange, Act, Assert):

```
[Fact]
public void Shipped_order_cannot_be_cancelled()
{
    var po = TestData.SubmittedOrder();          // Arrange
    po.TransitionTo(PurchaseOrderStatus.Accepted, null);
    po.TransitionTo(PurchaseOrderStatus.Shipped, null);
    Assert.Throws<InvalidOperationException>(() =>   // Act + Assert
        po.TransitionTo(PurchaseOrderStatus.Cancelled, null));
}
```

(Illustrative of the style; the real tests live in `tests/Catalogue.UnitTests`.)

Run them: `dotnet test` (all), or `dotnet test --filter FullyQualifiedName~PaymentTests` (one class). In VS Code use Test Explorer or the ".NET: Debug unit tests" launch profile.

11.2 CI pipeline (`.github/workflows/ci.yml`)

Runs on every pull request and every push to `main`, on `ubuntu-latest`:

1. Checkout and install .NET 10.
2. `dotnet restore`, `dotnet build -c Release`, `dotnet test`.
3. Frontend: `npm ci`, `vue-tsc -b` (type check), `vite build`.
4. Validate every `logicapps/*/workflow.json` is valid JSON.
5. `docker build` the API, worker and gateway images (proves the Dockerfiles work).

It **builds** images but does not push or deploy them. Section 12 adds that.

12. Deploying with Azure Container Registry (ACR)

Definitions first:

- **Docker image:** a sealed package of the app plus everything it needs (the .NET runtime, our DLLs). Built from a `Dockerfile`. Same image runs the same everywhere.
- **Container:** a running copy of an image.
- **Registry:** a store for images. **ACR** is Azure's private registry, so only your Azure resources can pull your images.
- **Container host:** something that runs images. We use **Azure Container Apps (ACA)**: it runs containers, gives HTTPS URLs, scales from 0 to N copies, and can scale on queue length. (Alternatives: App Service for Containers, AKS.)

The idea: build image, push to ACR, tell Container Apps to run that image.

12.1 Images we already have

Image	Dockerfile	What runs	Port
`catalogue-api`	`Dockerfile`	`dotnet Catalogue.Web.dll`	8080
`catalogue-worker`	`Dockerfile.worker`	`Catalogue.Workers` (queue + Event Hub consumers)	none

Image	Dockerfile	What runs	Port
`catalogue-gateway`	`Dockerfile.gateway`	YARP gateway	8080
`catalogue-web`	`web/catalogue-ui/Dockerfile`	nginx + built Vue `dist/`	80

Each .NET Dockerfile is **multi-stage**: stage 1 uses the big `mcr.microsoft.com/dotnet/sdk:10.0` image to `dotnet publish`; stage 2 copies only the output into the small `aspnet:10.0` runtime image. Result: small, secure images with no compiler inside.

12.2 One-time Azure setup (Azure CLI)

```
az login
az group create -n rg-catalogue -l uksouth

# Registry (name must be globally unique, letters and numbers only)
az acr create -g rg-catalogue -n acrcatalogue123 --sku Basic

# Container Apps environment (shared network + logs for all apps)
az extension add --name containerapp --upgrade
az containerapp env create -g rg-catalogue -n cae-catalogue -l uksouth
```

12.3 Build and push the images

Option A: build on your PC, push to ACR

```
az acr login -n acrcatalogue123          # docker now trusts the registry

TAG=$(git rev-parse --short HEAD)       # tag with the commit, never only
'latest'
ACR=acrcatalogue123.azurecr.io

docker build -t $ACR/catalogue-api:$TAG .
docker build -t $ACR/catalogue-worker:$TAG  -f Dockerfile.worker .
docker build -t $ACR/catalogue-gateway:$TAG -f Dockerfile.gateway .
docker build -t $ACR/catalogue-web:$TAG     web/catalogue-ui

docker push $ACR/catalogue-api:$TAG
docker push $ACR/catalogue-worker:$TAG
docker push $ACR/catalogue-gateway:$TAG
docker push $ACR/catalogue-web:$TAG
```

Option B: let Azure build it (no Docker needed on your PC)

```
az acr build -r acrcatalogue123 -t catalogue-api:$TAG .
az acr build -r acrcatalogue123 -t catalogue-worker:$TAG -f Dockerfile.worker
.
az acr build -r acrcatalogue123 -t catalogue-gateway:$TAG -f
Dockerfile.gateway .
az acr build -r acrcatalogue123 -t catalogue-web:$TAG web/catalogue-ui
```

Check: `az acr repository list -n acrcatalogue123 -o table`.

12.4 Run the images in Container Apps

```
# API: internal ingress (only reachable from the web/gateway apps)
az containerapp create -g rg-catalogue -n catalogue-api --environment cae-
catalogue \
  --image $ACR/catalogue-api:$TAG --registry-server $ACR --registry-identity
system \
  --ingress internal --target-port 8080 --min-replicas 1 --max-replicas 5 \
  --system-assigned \
  --env-vars ASPNETCORE_ENVIRONMENT=Production Worker__InProcess=false \
             ConnectionStrings__CatalogueDb=secretref:sql-conn

# Worker: no ingress, scales on queue length
az containerapp create -g rg-catalogue -n catalogue-worker --environment cae-
catalogue \
  --image $ACR/catalogue-worker:$TAG --registry-server $ACR --registry-
identity system \
  --min-replicas 0 --max-replicas 10 --system-assigned

# Web (nginx + Vue): public HTTPS
az containerapp create -g rg-catalogue -n catalogue-web --environment cae-
catalogue \
  --image $ACR/catalogue-web:$TAG --registry-server $ACR --registry-identity
system \
  --ingress external --target-port 80
```

Notes in plain words:

- `--registry-identity system` lets the app pull from ACR with its **managed identity** (it gets the `AcrPull` role), so no registry password is stored anywhere.
- Config uses **double underscore** for nesting: `ConnectionStrings__CatalogueDb` = `ConnectionStrings:CatalogueDb` in appsettings.
- `secretref:` points to a Container Apps secret (or a Key Vault reference); never put real secrets in plain env vars.
- nginx proxies `/api/` to `http://api:8080`; in Container Apps change that host to the API app's internal name (`http://catalogue-api`), see section 13.10.

Deploy a new version: build and push a new tag, then

```
az containerapp update -g rg-catalogue -n catalogue-api --image
$ACR/catalogue-api:$NEWTAG
```

Container Apps creates a new **revision** and moves traffic to it once it is healthy; roll back by re-activating the old revision.

12.5 Azure Functions and Logic Apps

These are not in the containers above:

- **Durable Functions:** `cd src/Catalogue.Functions` then `func azure functionapp publish func-catalogue` (to a Function App on the Flex Consumption or Premium plan with a Storage account). Or containerise it with `func init --docker-only` and push to ACR like the others.
- **Logic Apps Standard:** deploy the `logicapps/` folder as a ZIP to a Logic App (Standard) resource (`az logicapp deployment source config-zip`) or from VS Code's Azure extension.

12.6 Automate it: CD with GitHub Actions

Add a job after the existing `build-test` job in `.github/workflows/ci.yml`:

```
  deploy:
    needs: build-test
    if: github.ref == 'refs/heads/main'
    runs-on: ubuntu-latest
    permissions: { id-token: write, contents: read }   # OIDC, no stored
password
    steps:
      - uses: actions/checkout@v4
      - uses: azure/login@v2
        with:
          client-id: ${{ secrets.AZURE_CLIENT_ID }}
          tenant-id: ${{ secrets.AZURE_TENANT_ID }}
          subscription-id: ${{ secrets.AZURE_SUBSCRIPTION_ID }}
      - name: Build and push to ACR
        run: |
          az acr build -r acrcatalogue123 -t catalogue-api:${{ github.sha }}
.
          az acr build -r acrcatalogue123 -t catalogue-worker:${{ github.sha
}} -f Dockerfile.worker .
          az acr build -r acrcatalogue123 -t catalogue-web:${{ github.sha }}
web/catalogue-ui
      - name: Roll out
        run: |
```

```
          az containerapp update -g rg-catalogue -n catalogue-api    --image
acrcatalogue123.azurecr.io/catalogue-api:${{ github.sha }}
          az containerapp update -g rg-catalogue -n catalogue-worker --image
acrcatalogue123.azurecr.io/catalogue-worker:${{ github.sha }}
          az containerapp update -g rg-catalogue -n catalogue-web    --image
acrcatalogue123.azurecr.io/catalogue-web:${{ github.sha }}
```

OIDC (OpenID Connect) means GitHub proves its identity to Azure with a short-lived token, so no Azure password is saved in GitHub. Create it once with an Entra ID app registration + federated credential for this repo, and give it `AcrPush` on the registry and `Contributor` on the resource group.

13. Replacing the local setup with real Azure

The short version: most swaps are **configuration only**, because the code already talks to interfaces and reads connection strings. A few changes need **code**, mainly to log in with **Managed Identity** instead of keys, to use **migrations**, and to replace the local AI model. These code changes are described here only; they are **not** applied to the repository.

13.1 The full mapping

Local today	Azure service	Config change	Code change?
SQL Server LocalDB	**Azure SQL Database**	`ConnectionStrings__CatalogueDb`	Yes: migrations instead of EnsureCreated (13.2)
Azurite / `.storage` disk	**Azure Storage (Blob)**	`ConnectionStrings__BlobStorage`	Optional: Managed Identity (13.3)
In-memory queue	**Azure Service Bus**	`ServiceBus__Enabled=true`, `ConnectionStrings__ServiceBus`	Optional: Managed Identity (13.4)
In-memory Event Hub	**Azure Event Hubs**	`EventHub__Enabled=true`, `ConnectionStrings__EventHub`, `ConnectionStrings__CheckpointStorage`	No (13.5)

Local today	Azure service	Config change	Code change?
Redis container / none	**Azure Cache for Redis** (or Azure Managed Redis)	`ConnectionStrings__Redis`	No (13.6)
appsettings / user-secrets	**Azure Key Vault**	`KeyVault__Uri`	Yes: 3 lines (13.7)
Aspire dashboard (OTLP)	**Application Insights**	`APPLICATIONINSIGHTS_CONNECTION_STRING`	Yes: 2 lines (13.8)
Foundry Local (your GPU)	**Azure OpenAI / Azure AI Foundry**	`Agent__Provider=AzureOpenAI` + endpoint, deployment	Yes: new chat client (13.9)
Vite dev server	Container Apps (nginx) or **Static Web Apps**	`API_UPSTREAM`	Yes: nginx template (13.10)
`func start` on :7072	**Azure Functions** (Durable)	`AzureWebJobsStorage`, connection strings	Yes: secure the HTTP trigger (13.12)
Logic Apps on Core Tools	**Logic Apps Standard**	app settings	Yes: remove hard-coded URL and login (13.13)
Own JWT with dev key	Own JWT with key in Key Vault (or **Microsoft Entra ID**)	`Jwt__SigningKey`, `Jwt__Issuer`	No for Key Vault

Managed Identity in one line: Azure gives each Container App an identity; you grant that identity a role (e.g. "Storage Blob Data Contributor"), and the app logs in with `new DefaultAzureCredential()`. No keys or passwords in config at all. Locally, the same `DefaultAzureCredential` uses your `az login`, so the code is identical on both.

Package needed for any Managed Identity change: `dotnet add src/Catalogue.Infrastructure package Azure.Identity`.

13.2 Database: Azure SQL

Create it:

```
az sql server create -g rg-catalogue -n sql-catalogue -l uksouth \
  --enable-ad-only-auth --external-admin-principal-type User \
  --external-admin-name you@yourtenant.com --external-admin-sid <your-object-
id>
az sql db create -g rg-catalogue -s sql-catalogue -n CatalogueDb --service-
objective S1
az sql server firewall-rule create -g rg-catalogue -s sql-catalogue -n azure
--start-ip-address 0.0.0.0 --end-ip-address 0.0.0.0
```

Connection string (no password, uses the app's managed identity):

```
Server=tcp:sql-
catalogue.database.windows.net,1433;Database=CatalogueDb;Authentication=Activ
e Directory Default;Encrypt=True;
```

Give the app access (run in the database as the Entra admin):

```
CREATE USER [catalogue-api] FROM EXTERNAL PROVIDER;   -- the Container App's
name
ALTER ROLE db_datareader ADD MEMBER [catalogue-api];
ALTER ROLE db_datawriter ADD MEMBER [catalogue-api];
-- repeat for catalogue-worker and the Function App
```

This works for EF Core **and** `SqlBulkCatalogueWriter`, because both use `Microsoft.Data.SqlClient`, which understands `Authentication=Active Directory Default`.

Code change: migrations instead of EnsureCreated. `EnsureCreated` + `SchemaEvolver` cannot rename or drop columns and runs DDL from every app at startup. In production the schema should change once, from the pipeline.

1. Create the first migration (one time):

```
dotnet tool install -g dotnet-ef
dotnet add src/Catalogue.Web package Microsoft.EntityFrameworkCore.Design
dotnet ef migrations add InitialCreate -p src/Catalogue.Infrastructure -s
src/Catalogue.Web
```

2. Change `SeedAsync` in `Infrastructure/DependencyInjection.cs`:

```
public static async Task SeedAsync(this CatalogueDbContext db, bool
useMigrations = false, CancellationToken ct = default)
{
    if (useMigrations)
        await db.Database.MigrateAsync(ct);        // applies pending
migrations, never guesses
    else
    {
        await db.Database.EnsureCreatedAsync(ct);  // local learning path
(unchanged)
        await SchemaEvolver.EvolveAsync(db, ct);
    }
    await ApplySchemaPatchesAsync(db, ct);
    // ... seeding unchanged
}
```

3. In `Catalogue.Web/Program.cs`: `await db.SeedAsync(app.Configuration.GetValue<bool>("Database:UseMigrations"));` and set `Database__UseMigrations=true` in Azure.
4. Remove the `EnsureCreatedAsync()` calls in `Catalogue.Workers/Program.cs` and `Catalogue.Functions/FunctionServices.cs` (only the API owns the schema).
5. Best practice: build a migration bundle in CI (`dotnet ef migrations bundle -p src/Catalogue.Infrastructure -s src/Catalogue.Web -o efbundle`) and run `./efbundle --connection "$SQL_CONN"` before rolling out new images; then the app identity needs no DDL rights.

13.3 Files: Azure Blob Storage

Zero-code option: create a storage account and set `ConnectionStrings__BlobStorage` to its connection string (kept in Key Vault). `StorageSelection` sees a reachable Blob endpoint and uses `BlobFileStorage` / `BlobMediaStorage`. Signed media URLs keep working unchanged.

```
az storage account create -g rg-catalogue -n stcatalogue123 -l uksouth --sku
Standard_LRS --allow-blob-public-access false
```

Recommended: Managed Identity (code change). Add constructors that take a `BlobServiceClient`:

```
// Storage/FileStorages.cs  (same idea in MediaStorages.cs for
BlobMediaStorage)
public BlobFileStorage(BlobServiceClient service, string container)
{
    _container = service.GetBlobContainerClient(container);
    _ensure = new Lazy<Task>(() =>
_container.CreateIfNotExistsAsync(PublicAccessType.None));
}
```

Then in `AddCatalogueInfrastructure`, before the existing storage lines:

```
if (config["BlobStorage:ServiceUri"] is { Length: > 0 } blobUri)   //
https://stcatalogue123.blob.core.windows.net
{
    var blob = new BlobServiceClient(new Uri(blobUri), new
DefaultAzureCredential(), BlobOptions.Create());
    services.AddSingleton<IFileStorage>(new BlobFileStorage(blob,
config["FileStorage:Container"] ?? "catalogue-raw"));
    services.AddSingleton<Application.Media.IMediaStorage>(new
BlobMediaStorage(blob, config["Media:Container"] ?? "catalogue-media"));
}
else
{
    // existing StorageSelection code (Azurite or .storage disk) stays for
local development
}
```

Grant the role: `az role assignment create --assignee <app-principal-id> --role "Storage Blob Data Contributor" --scope <storage-account-id>`.

Important: in a container the local disk is **temporary** (lost on restart and not shared between copies), so in Azure the disk fallback must never be used. With the `ServiceUri` branch above there is no fallback; with the connection-string option, watch for the startup warning "falling back to local disk".

13.4 Queue: Azure Service Bus

```
az servicebus namespace create -g rg-catalogue -n sb-catalogue --sku Standard
az servicebus queue create -g rg-catalogue --namespace-name sb-catalogue -n
catalogue-imports \
  --enable-duplicate-detection true --duplicate-detection-history-time-window
PT10M \
  --max-delivery-count 10 --lock-duration PT5M
```

Config on API and worker: `ServiceBus__Enabled=true`, `ServiceBus__QueueName=catalogue-imports`, `ConnectionStrings__ServiceBus=<from Key Vault>`. On the **API** also set `Worker__InProcess=false` so the import work moves to `catalogue-worker`.

Managed Identity (code change in `Messaging/ImportQueues.cs`):

```
var clientOptions = new ServiceBusClientOptions { RetryOptions = /* unchanged
*/ };
_client = config["ServiceBus:Namespace"] is { Length: > 0 } ns            //
sb-catalogue.servicebus.windows.net
    ? new ServiceBusClient(ns, new DefaultAzureCredential(), clientOptions)
    : new ServiceBusClient(config.GetConnectionString("ServiceBus"),
clientOptions);
```

Roles: **Azure Service Bus Data Sender** for the API, **Data Receiver** for the worker. Let the worker scale on queue length:

```
az containerapp update -g rg-catalogue -n catalogue-worker \
  --scale-rule-name imports --scale-rule-type azure-servicebus \
  --scale-rule-metadata queueName=catalogue-imports namespace=sb-catalogue
messageCount=5 \
  --scale-rule-identity system
```

13.5 Streaming: Azure Event Hubs

```
az eventhubs namespace create -g rg-catalogue -n evh-catalogue --sku Standard
az eventhubs eventhub create -g rg-catalogue --namespace-name evh-catalogue -
n catalogue-edi --partition-count 4
az eventhubs eventhub consumer-group create -g rg-catalogue --namespace-name
evh-catalogue --eventhub-name catalogue-edi -n catalogue-ingest
```

Config: `EventHub__Enabled=true`, `ConnectionStrings__EventHub=<namespace connection string>`, `ConnectionStrings__CheckpointStorage=<storage connection string>` (checkpoints go to the `eventhub-checkpoints` container). On the API set `EventHub__ConsumerInProcess=false`; on the worker set it to `true`, so every worker copy joins the consumer group and partitions are shared out automatically. No code change.

13.6 Cache: Azure Cache for Redis

Set `ConnectionStrings__Redis` to `redis-catalogue.redis.cache.windows.net:6380,password=<key>,ssl=True,abortConnect=False`. `Program.cs` already switches HybridCache to use Redis as L2 when this is set, wrapped in `ResilientDistributedCache` so a Redis outage only slows things down. No code change. This matters in Azure: with several API copies, Redis makes them share one cache and tag invalidation works across all of them.

13.7 Secrets: Azure Key Vault (code change)

```
dotnet add src/Catalogue.Web package
Azure.Extensions.AspNetCore.Configuration.Secrets
dotnet add src/Catalogue.Web package Azure.Identity
```

In `Catalogue.Web/Program.cs`, right after `WebApplication.CreateBuilder(args)` (same lines in `Catalogue.Workers/Program.cs`):

```
if (builder.Configuration["KeyVault:Uri"] is { Length: > 0 } kv)
    builder.Configuration.AddAzureKeyVault(new Uri(kv), new
DefaultAzureCredential());
```

Secret names use `--` where config uses `:`:

Key Vault secret	Becomes config key
`ConnectionStrings--CatalogueDb`	`ConnectionStrings:CatalogueDb`
`ConnectionStrings--BlobStorage`, `--ServiceBus`, `--EventHub`, `--CheckpointStorage`, `--Redis`	the matching connection strings
`Jwt--SigningKey`	JWT signing key (at least 32 random bytes)
`Media--SigningKey`	key for signed image URLs
`Payments--Stripe--SecretKey`, `Payments--Stripe--WebhookSecret`	Stripe keys

Grant the apps **Key Vault Secrets User**. Now no secret lives in appsettings, images, or GitHub.

13.8 Monitoring: Application Insights (code change)

Today telemetry is exported only when `OTEL_EXPORTER_OTLP_ENDPOINT` is set. Add the Azure Monitor exporter in `Catalogue.ServiceDefaults/ServiceDefaults.cs`:

```
// dotnet add src/Catalogue.ServiceDefaults package
Azure.Monitor.OpenTelemetry.Exporter
if
(!string.IsNullOrWhiteSpace(builder.Configuration["OTEL_EXPORTER_OTLP_ENDPOIN
T"]))
    otel.UseOtlpExporter();
if
(!string.IsNullOrWhiteSpace(builder.Configuration["APPLICATIONINSIGHTS_CONNEC
TION_STRING"]))
    otel.UseAzureMonitorExporter();   // traces, metrics and logs to
Application Insights
```

Set `APPLICATIONINSIGHTS_CONNECTION_STRING` on every app. Because every service already tags spans with the **correlation id**, one search in App Insights shows the whole journey: browser, API, Service Bus, worker, SQL. Console logs are also collected by Container Apps into Log Analytics; the rolling file under `logs/` is lost on container restart, so do not rely on it in Azure.

13.9 AI model: Foundry Local to Azure OpenAI (code change)

Foundry Local runs on **your** GPU and cannot run inside a cloud container. In Azure use **Azure OpenAI** (inside Azure AI Foundry): deploy a chat model with tool calling (for example a GPT-4.1-mini class deployment). Hosted models return **native tool calls**, so our text tool-call parser is not needed there.

Step 1: a small runtime interface so the assistant does not depend on Foundry Local directly (`Catalogue.Agents`):

```
public interface IModelRuntime
{
    FoundryLocalOptions Options { get; }            // MaxOutputTokens,
Temperature
    Task<FoundryConnection> EnsureReadyAsync(CancellationToken ct);
    Task<FoundryLocalStatus> GetStatusAsync(bool refresh, CancellationToken
ct);
}

// FoundryLocalRuntime already has these members: just add ": IModelRuntime".

public sealed class AzureOpenAIRuntime(Uri endpoint, string deployment,
FoundryLocalOptions options) : IModelRuntime
{
    public FoundryLocalOptions Options => options;
    public Task<FoundryConnection> EnsureReadyAsync(CancellationToken ct) =>
Task.FromResult(new FoundryConnection(endpoint, deployment));
    public Task<FoundryLocalStatus> GetStatusAsync(bool refresh,
CancellationToken ct) => Task.FromResult(new FoundryLocalStatus(
        true, true, endpoint.ToString(), deployment, deployment, true, true,
true, "Azure OpenAI", null, DateTimeOffset.UtcNow));
}
```

Then change the constructor parameter type from `FoundryLocalRuntime` to `IModelRuntime` in `CatalogueAssistant` , `OperationsReviewRunner` , `FoundryLocalHealthCheck` and the `/api/agent/status` endpoint.

Step 2: register the Azure chat client in `AgentServiceCollectionExtensions.AddCatalogueAgents` :

```
// dotnet add src/Catalogue.Agents package Azure.AI.OpenAI
// dotnet add src/Catalogue.Agents package Microsoft.Extensions.AI.OpenAI
if (config["Agent:Provider"] == "AzureOpenAI")
{
    var endpoint = new Uri(config["Agent:AzureOpenAI:Endpoint"]!);   //
https://<name>.openai.azure.com/
    var deployment = config["Agent:AzureOpenAI:Deployment"]!;
    services.AddSingleton<IModelRuntime>(new AzureOpenAIRuntime(endpoint,
deployment, foundry));
    services.AddKeyedSingleton<IChatClient>(AgentServiceKeys.Model, (sp, _)
=>
        new AzureOpenAIClient(endpoint, new DefaultAzureCredential())
            .GetChatClient(deployment).AsIChatClient()          // native
tool calls: no TextToolCallChatClient
            .AsBuilder()
            .UseFunctionInvocation(sp.GetRequiredService<ILoggerFactory>(), f
=>
            {
                f.MaximumIterationsPerRequest = agent.MaxToolCallsPerRun + 1;
                f.MaximumConsecutiveErrorsPerRequest = 2;
                f.AllowConcurrentInvocation = false;
            })
            .UseOpenTelemetry(sp.GetRequiredService<ILoggerFactory>(),
CatalogueAssistant.SourceName)
            .Build(sp));
}
else
{
    services.AddSingleton<IModelRuntime>(sp =>
sp.GetRequiredService<FoundryLocalRuntime>());
    // existing Foundry Local registration unchanged
}
```

Config: `Agent__Provider=AzureOpenAI`, `Agent__AzureOpenAI__Endpoint`, `Agent__AzureOpenAI__Deployment`, `Agent__FoundryLocal__Enabled=false`. Role for the API identity: **Cognitive Services OpenAI User**. Everything else (tools, permissions, proposals, workflow, SSE) is unchanged, which is the payoff of using `IChatClient`. Note for interviews: with Azure, prompts and tool results leave your PC (they stay in your Azure tenant), and you pay per token.

13.10 Frontend: nginx in Azure (code change)

`web/catalogue-ui/nginx.conf` hard-codes `proxy_pass http://api:8080;` (the docker-compose name) and sends the browser's Host header. In Container Apps the API has a different name, and Container Apps routes by Host, so:

1. Rename `nginx.conf` to `default.conf.template` and change the `/api/` block:

```
location /api/ {
    proxy_pass ${API_UPSTREAM};                 # filled from an env var at
container start
    proxy_set_header X-Forwarded-For $proxy_add_x_forwarded_for;
    proxy_set_header X-Forwarded-Proto $scheme;  # (removed: proxy_set_header
Host $host)
    proxy_request_buffering off;
    proxy_read_timeout 600s;
}
```

2. In `web/catalogue-ui/Dockerfile` replace the config copy line:

```
COPY default.conf.template /etc/nginx/templates/default.conf.template
ENV API_UPSTREAM=http://api:8080
```

The official nginx image substitutes environment variables in `/etc/nginx/templates/*.template` at start-up. In Azure set `API_UPSTREAM=http://catalogue-api` (internal ingress with insecure HTTP allowed inside the environment) or the API's internal HTTPS address.

Alternative: host the built `dist/` on **Azure Static Web Apps** and link the API as its backend; then set `AllowedOrigins` if the origins differ.

13.11 Small but necessary API settings

Setting	Why	Value in Azure
`ASPNETCORE_ENVIRONMENT`	Turns off Swagger and dev defaults	`Production`

Setting	Why	Value in Azure
`Payments__ReturnBaseUrl`	Where Stripe sends the buyer back	`https://catalogue-web.<env>.azurecontainerapps.io`
Stripe dashboard webhook	Stripe must reach us	`https://<web-url>/api/payments/webhooks/stripe`
`Payments__Simulated__Enabled`	Fake payments only for dev	`false`
`AllowedOrigins__0`	Only if the SPA is on another origin	the SPA URL
`Jwt__Issuer`	Token issuer name	e.g. `catalogue-prod`

Code change: forwarded headers. Behind nginx and the Container Apps proxy, the API sees every request coming from the proxy's IP over http. Rate limits by IP would then treat all users as one. Add to `Program.cs`:

```
builder.Services.Configure<ForwardedHeadersOptions>(o =>
{
    o.ForwardedHeaders = ForwardedHeaders.XForwardedFor |
ForwardedHeaders.XForwardedProto;
    o.KnownIPNetworks.Clear(); o.KnownProxies.Clear();   // trust the
platform proxy (API is internal-only)
});
// first line after var app = builder.Build();
app.UseForwardedHeaders();
```

Scaling note: AI runs and their SSE streams live in the memory of one API copy (`AgentRunCoordinator`). With more than one API replica, either turn on sticky sessions (`az containerapp ingress sticky-sessions set -n catalogue-api -g rg-catalogue --affinity sticky`) or keep the API at one replica until runs are moved to a shared store (Redis or Service Bus).

13.12 Durable Functions in Azure

```
az functionapp create -g rg-catalogue -n func-catalogue --storage-account
stcatalogue123 \
  --flexconsumption-location uksouth --runtime dotnet-isolated --runtime-
version 10.0
az functionapp config appsettings set -g rg-catalogue -n func-catalogue --
settings \
  "ConnectionStrings__CatalogueDb=@Microsoft.KeyVault(SecretUri=https://kv-
catalogue.vault.azure.net/secrets/ConnectionStrings--CatalogueDb/)"
cd src/Catalogue.Functions && func azure functionapp publish func-catalogue
```

- `AzureWebJobsStorage` now points at the real storage account, so the orchestration history (tables and queues) lives there instead of Azurite. Nothing else in `host.json` changes.
- **Code change:** `SyncHttpClient` uses `AuthorizationLevel.Anonymous`, so anyone could start a sync. Change it to `AuthorizationLevel.Function` (callers then need the function key) or put the function behind the API.
- Optional: switch the Durable backend to the managed **Durable Task Scheduler** for better throughput and a built-in monitoring dashboard.

13.13 Logic Apps Standard in Azure (code change)

The workflows call `http://localhost:5080` and log in with the hard-coded demo admin account. Before deploying:

1. Replace the base URL with an app setting: `"uri": "@{appsetting('CatalogueApiBaseUrl')}/api/auth/login"`.
2. Replace the username and password with `@{appsetting('CatalogueApiUser')}` / `@{appsetting('CatalogueApiPassword')}`, and make those app settings **Key Vault references**. Better still, create a dedicated read-only service account.
3. In `import-monitor`, add the real alert action (Office 365 Outlook "Send an email" or Teams "Post message") in place of the comment; that adds a managed connection to `connections.json`.
4. Deploy: `cd logicapps && zip -r ../la.zip . && az logicapp deployment source config-zip -g rg-catalogue -n la-catalogue --src ../la.zip`.

13.14 Final Azure picture

```
Internet --HTTPS--> catalogue-web (nginx + Vue)        [Container Apps,
external]
                       | /api/*
                       v
                    catalogue-api  ----> Azure SQL (Managed Identity)
                    [internal]     ----> Blob Storage (images, raw files)
                       |           ----> Azure Cache for Redis
                       |           ----> Azure OpenAI (assistant)
                       |           ----> Key Vault (secrets)    ----> App
Insights (all telemetry)
                       v
             Service Bus queue / Event Hubs
                       v
                catalogue-worker (scales 0..N on queue length)

func-catalogue (Durable supplier sync, every 30 min)   la-catalogue (Logic
Apps monitor/notify)
All images pulled from ACR by managed identity; deployed by GitHub Actions
(OIDC).
```

14. How to present this project

14.1 The 30-second pitch

> "I built a B2B catalogue and marketplace on **.NET 10 and Vue 3**. Suppliers load products by CSV, EDI or API, up to 2 GB files streamed through **Event Hubs** and bulk-merged into **SQL Server**. Buyers compare every seller's price for their account, and checkout creates one purchase order per seller with **Stripe** payments. Long jobs run in **Service Bus workers** and **Durable Functions**, and an **AI agent** built with Microsoft Agent Framework answers operations questions using read-only tools, with every change approved by a human. It runs locally on emulators and moves to **Azure** (Container Apps, Azure SQL, Blob, Service Bus, Key Vault, App Insights) mostly by configuration."

14.2 The 5-minute walkthrough (demo order)

Step	Show	Say
1	Section 1 diagram	Five layers, Clean Architecture, interfaces everywhere
2	Login, then DevTools Network tab	JWT in memory, refresh cookie HttpOnly, correlation id header
3	Upload a CSV in Imports	202 Accepted, queue, worker, row-level errors
4	Event Hub screen with a big EDI file	Partitions, checkpoints, chunk ledger = idempotent
5	Approvals	Price changes over a threshold need a human
6	Basket, then Orders, then Pay	Server re-prices, one PO per seller, webhook HMAC, no double payment
7	Analytics, switch 7d to 1y	Buckets, previous period, cache
8	Assistant: "Which orders are waiting?"	Tool calls appear live (SSE); agent can only propose
9	Section 13 table	How each local piece becomes an Azure service

14.3 Questions you will be asked, with short answers

Question	Answer
Why minimal APIs, not controllers?	Less ceremony, faster startup, same features (DI, auth, filters, OpenAPI). Grouped per feature in `Endpoints/*`.
How do you stop a payment being taken twice?	Pending attempt is reused; `Payment.Succeed` is idempotent; a **filtered unique index** allows one success per order even under a race.
What if the worker crashes mid-import?	Service Bus PeekLock redelivers; Event Hubs resumes from the checkpoint; the chunk ledger skips work already done; startup sweep marks stuck imports reprocessable.

Question	Answer
What is Durable Functions and why use it?	Code-first stateful workflows. The orchestrator is replayed from saved history, so a 30-minute fan-out sync survives restarts and retries each step.
Orchestrator vs activity?	Orchestrator decides (deterministic, no I/O); activity does the I/O and can be retried.
How is the AI kept safe?	Same permissions as the user, read-only tools, budgets and timeouts, and writes only through human-approved proposals that are re-authorised at approval time.
Why a local model?	Private data stays on the machine and there is no token cost; swapping to Azure OpenAI is one DI registration because everything uses `IChatClient`.
How do you handle secrets?	Locally: user-secrets and dev defaults. Azure: Key Vault + Managed Identity, so no secret is in code, images or GitHub.
How does it scale?	Stateless API copies behind Container Apps; Redis shares the cache; workers scale on queue length; Event Hub partitions spread over workers.
How do you trace a bug?	Every request has a correlation id shown in the UI error, written in logs and audit, and carried on queue messages and OpenTelemetry spans.
What would you improve next?	EF migrations in the pipeline, Managed Identity everywhere, move agent runs to a shared store for multi-replica SSE, Entra ID sign-in, integration tests against real SQL with Testcontainers.

14.4 Where to look in the code for each claim

Claim	File
Startup and middleware order	`src/Catalogue.Web/Program.cs`
DI choices (Blob vs disk, Service Bus vs memory, Stripe vs simulated)	`src/Catalogue.Infrastructure/DependencyInjection.cs`
Tables, indexes, filtered unique indexes	`src/Catalogue.Infrastructure/Persistence/CatalogueDbContext.cs`
Bulk MERGE + chunk ledger	`src/Catalogue.Infrastructure/Streaming/SqlBulkCatalogueWriter.cs`
Durable Functions	`src/Catalogue.Functions/SupplierSyncFunctions.cs`, `host.json`
Pricing rules	`src/Catalogue.Application/Pricing.cs`, `Marketplace.cs`
Payments	`src/Catalogue.Application/Payments.cs`, `Infrastructure/Payments/PaymentGateways.cs`
AI agent	`src/Catalogue.Agents/*`
Frontend API client	`web/catalogue-ui/src/api.ts`
Every endpoint explained	`docs/api/README.md`
Step-by-step debugging	the "Debugging Guide" doc